1. กำหนด Node และ Relationship

ใช้โครงสร้างดังนี้

- Node: Student, Club, Interest
- Relationship: FRIEND_OF, MEMBER_OF, INTERESTED_IN, FOCUSES_ON
- เป้าหมาย: แนะนำชมรมที่เพื่อนเป็นสมาชิก หรือชมรมที่ตรงกับความสนใจของนักเรียน

Pattern หลัก:

(me:Student)-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]->(club:Club)

คำอธิบาย: *เริ่มจากนักเรียน แล้วเดินความสัมพันธ์ไปหาเพื่อน จากนั้นดูว่าเพื่อนแต่ละคนเป็นสมาชิกชมรมใด*

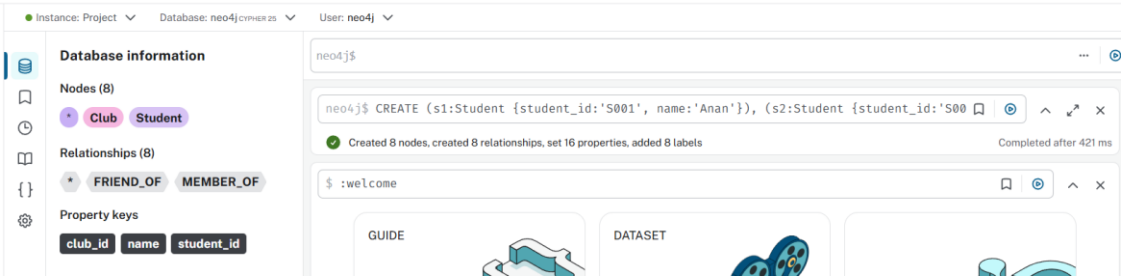
2. สร้างข้อมูลตัวอย่าง

```
CREATE
(s1:Student {student_id:'S001', name:'Anan'}),
(s2:Student {student_id:'S002', name:'Beam'}),
(s3:Student {student_id:'S003', name:'Cake'}),
(s4:Student {student_id:'S004', name:'Dear'}),
(s5:Student {student_id:'S005', name:'Earth'}),

(c1:Club {club_id:'C001', name:'Coding Club'}),
(c2:Club {club_id:'C002', name:'Music Club'}),
(c3:Club {club_id:'C003', name:'Sports Club'}),

(s1)-[:FRIEND_OF]->(s2),
(s1)-[:FRIEND_OF]->(s3),
(s1)-[:FRIEND_OF]->(s4),

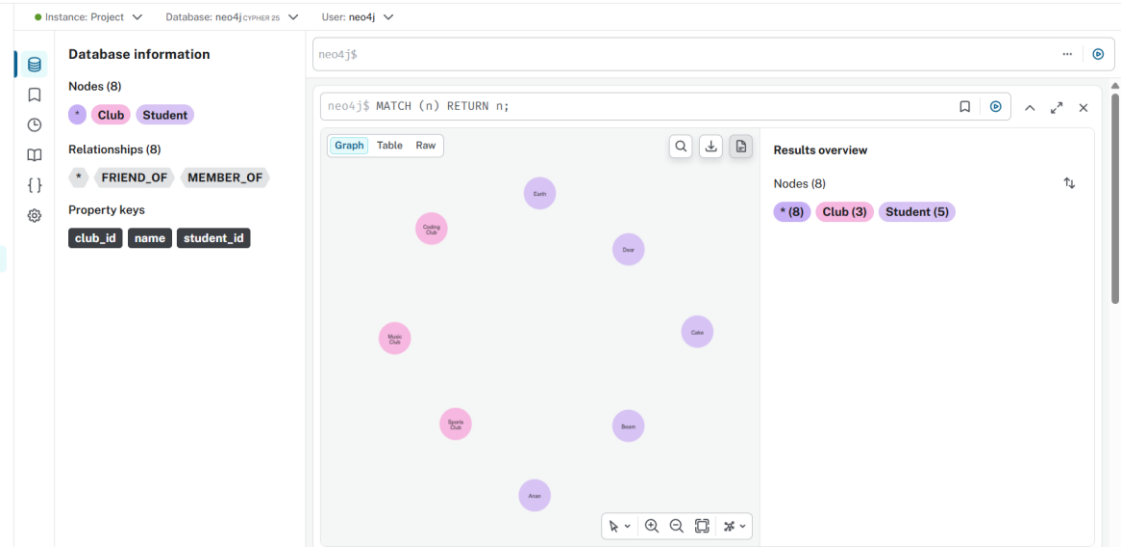
(s2)-[:MEMBER_OF]->(c1),
(s3)-[:MEMBER_OF]->(c1),
(s4)-[:MEMBER_OF]->(c2),
(s5)-[:MEMBER_OF]->(c3),
(s1)-[:MEMBER_OF]->(c3);
```



คำอธิบาย: *คำสั่งนี้สร้างนักเรียน 5 คน ชมรม 3 ชมรม รวมถึงกำหนดว่าใครเป็นเพื่อนกับใครและใครเป็นสมาชิกชมรมใด*

3. ตรวจสอบข้อมูลทั้งหมด

```
MATCH (n)
RETURN n;
```

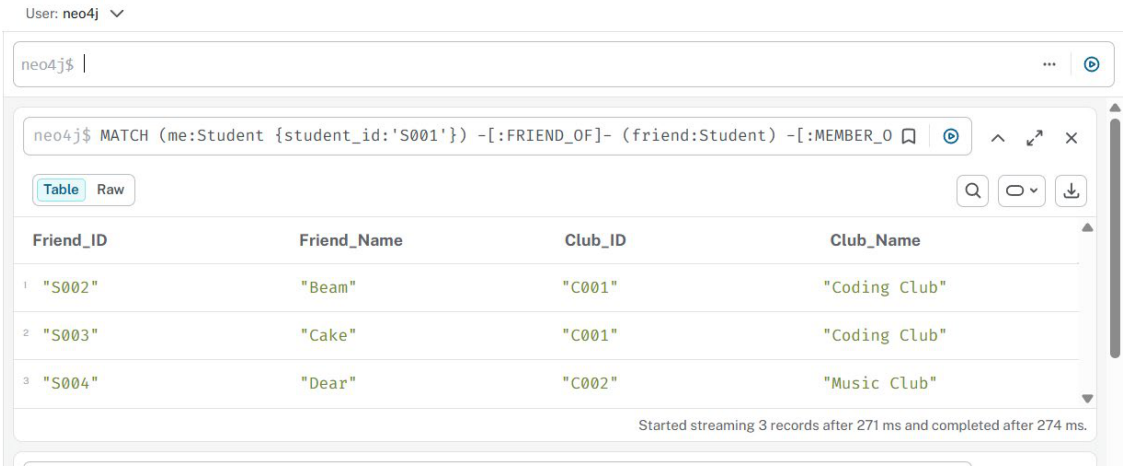


คำอธิบาย: ผลลัพธ์แสดงนักเรียน 5 คน ชมรม 3 ชมรม และความสัมพันธ์ระหว่างนักเรียนกับชมรม แสดงว่าระบบสร้างข้อมูลสำเร็จ

4. ดูชมรมของเพื่อนนักเรียน S001

```
MATCH (me:Student {student_id:'S001'})
  -[:FRIEND_OF]-
  (friend:Student)
  -[:MEMBER_OF]->
  (club:Club)
RETURN friend.student_id AS Friend_ID,
       friend.name AS Friend_Name,
       club.club_id AS Club_ID,
```

club.name AS Club_Name;



คำอธิบาย: ค้นหานักเรียนรหัส S001 จากนั้นเดินผ่านความสัมพันธ์ FRIEND_OF เพื่อหาเพื่อน และเดินต่อผ่าน MEMBER_OF เพื่อดูว่าเพื่อนอยู่ชมรมใด

5. นับจำนวนเพื่อนในแต่ละชมรมเพื่อให้คะแนน

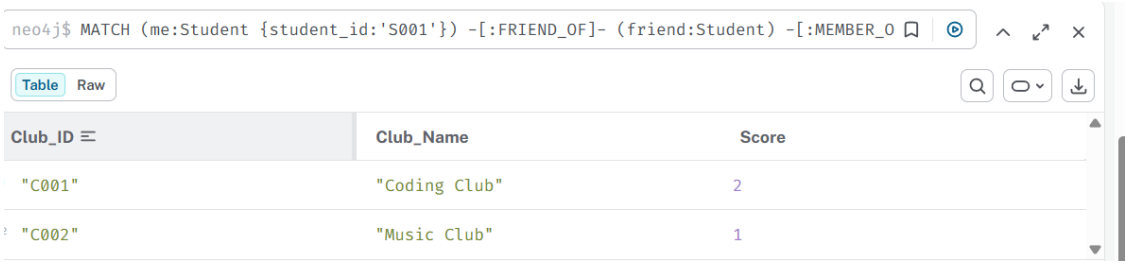
```
MATCH (me:Student {student_id:'S001'})
  -[:FRIEND_OF]-
  (friend:Student)
  -[:MEMBER_OF]->
  (club:Club)
RETURN club.club_id AS Club_ID,
       club.name AS Club_Name,
       count(DISTINCT friend) AS Score
ORDER BY Score DESC;
```



คำอธิบาย: count(DISTINCT friend) ใช้นับจำนวนเพื่อนที่เป็นสมาชิกของแต่ละชมรม โดยชมรมที่มีเพื่อนอยู่มากกว่าจะได้คะแนนสูงกว่า

6. ตัดชมรมที่ S001 เป็นสมาชิกอยู่แล้ว

```
MATCH (me:Student {student_id:'S001'})
  -[:FRIEND_OF]-
  (friend:Student)
  -[:MEMBER_OF]->
  (club:Club)
WHERE NOT (me)-[:MEMBER_OF]->(club)
RETURN club.club_id AS Club_ID,
       club.name AS Recommended_Club,
       count(DISTINCT friend) AS Score
ORDER BY Score DESC;
```



คำอธิบาย: เงื่อนไข WHERE NOT ใช้ตัดชมรมที่ S001 เป็นสมาชิกอยู่แล้วออก เพื่อไม่ให้ระบบแนะนำชมรมเดิมซ้ำ ในตัวอย่างนี้ S001 เป็นสมาชิกอยู่แล้วเฉพาะ Sports Club (C003) ซึ่งไม่มีเพื่อนของ S001 คนใดเป็นสมาชิกอยู่ก่อนแล้ว
เงื่อนไขนี้จึงไม่มีผลเปลี่ยนแปลงผลลัพธ์ในกรณีนี้ แต่ยังจำเป็นสำหรับกรณีทั่วไปที่ชมรมของเพื่อนอาจซ้ำกับชมรมที่ตนเองเป็นสมาชิกอยู่แล้ว

7. แสดงชมรมแนะนำอันดับ 1

```
MATCH (me:Student {student_id:'S001'})
  -[:FRIEND_OF]-
  (friend:Student)
  -[:MEMBER_OF]->
  (club:Club)
WHERE NOT (me)-[:MEMBER_OF]->(club)
RETURN club.club_id AS Club_ID,
       club.name AS Recommended_Club,
       count(DISTINCT friend) AS Score
ORDER BY Score DESC
LIMIT 1;
```



คำอธิบายผลลัพธ์:

ระบบแนะนำ Coding Club เป็นอันดับ 1 เนื่องจากมีเพื่อนของนักเรียน S001 เป็นสมาชิกจำนวน 2 คน (Beam และ Cake) มากกว่า Music Club ซึ่งมีเพื่อนเป็นสมาชิกเพียง 1 คน (Dear)

ส่วน Sports Club ไม่ปรากฏในผลลัพธ์การแนะนำ เนื่องจากไม่มีเพื่อนคนใดของ S001 (Beam, Cake, Dear) เป็นสมาชิกของ Sports Club เลย — มีเพียง Earth ที่เป็นสมาชิก Sports Club แต่ Earth ไม่ได้เป็นเพื่อนของ S001 ในข้อมูลชุดนี้ ส่วนเงื่อนไข WHERE NOT (me)-[:MEMBER_OF]->(club) ในขั้นตอนที่ 6 มีไว้เพื่อป้องกันไม่ให้ระบบแนะนำชมรมที่ S001 เป็นสมาชิกอยู่แล้วซ้ำ
ในกรณีทั่วไปที่ชมรมนั้นอาจมีเพื่อนเป็นสมาชิกอยู่ด้วย ซึ่งในชุดข้อมูลตัวอย่างนี้ไม่มีชมรมใดตรงกับเงื่อนไขดังกล่าวพอดี
ผลลัพธ์ก่อนและหลังใส่เงื่อนไข WHERE NOT จึงเหมือนกัน